ES-CS301 — Data Structure & Algorithms

Semester III • 3L:0T:4P • 3 Credits

Module 4 Exam Notes: Sorting Algorithms, Hashing Techniques and Graph Theory

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1 Classification and Criteria of Sorting Algorithms **2**

2 Elementary Sorting Algorithms **2**

2.1 Selection Sort 3

2.2 Bubble Sort 3

2.3 Insertion Sort 3

3 Advanced Divide-and-Conquer & Tree-Based Sorting **3**

3.1 Quick Sort (Partition-Exchange Sort) 4

3.2 Merge Sort 4

3.3 Heap Sort 5

3.4 Comprehensive Sorting Comparison Matrix 6

4 Hashing Techniques and Collision Resolution **6**

4.1 Standard Hash Functions 6

4.2 Collision Resolution Techniques 7

5 Graph Theory, Representations and Traversals **7**

5.1 Graph Storage Representations 8

5.2 Graph Search and Traversal Algorithms 8

1. Classification and Criteria of Sorting Algorithms

Definition: Sorting Problem and Taxonomic Criteria

Given a sequence of n records $R_1, R_2, \dots, R_n$ with keys $K_1, K_2, \dots, K_n$, **sorting** is the algorithmic permutation of these records into an ordered sequence $R_{\pi(1)}, R_{\pi(2)}, \dots, R_{\pi(n)}$ such that:

$$K_{\pi(1)} \leq K_{\pi(2)} \leq \dots \leq K_{\pi(n)} \tag{1}$$

Core Taxonomic Criteria:

1. **Internal vs External Sorting:** Internal sorting holds the entire dataset concurrently in main RAM. External sorting processes massive datasets residing on secondary storage (hard drives, tapes) using phased multi-way merge passes.
2. **Space Efficiency (In-Place vs Out-of-Place):** An in-place algorithm requires strictly $\mathcal{O}(1)$ auxiliary space beyond the input buffer (e.g., Insertion Sort, Heap Sort). Out-of-place algorithms require $\mathcal{O}(n)$ auxiliary storage (e.g., Merge Sort).
3. **Stability:** A sorting algorithm is **stable** if records with identical keys preserve their relative initial input ordering:

$$K_i = K_j \text{ with } i < j \implies \pi(i) < \pi(j) \tag{2}$$

4. **Adaptivity:** An algorithm is adaptive if its runtime decreases when the input array is already partially or completely sorted (e.g., Bubble Sort with swap flag, Insertion Sort).

Theoretical Lower Bound for Comparison-Based Sorting

Any deterministic comparison-based sorting algorithm can be modeled as a binary decision tree where each internal node represents a comparison $A[i] \leq A[j]$ and each leaf represents one of the $n!$ possible permutations of the input.

For a binary tree of height h with at least $n!$ leaves:

$$2^h \geq n! \implies h \geq \log_2(n!) \tag{3}$$

Applying Stirling's approximation ($\ln(n!) \approx n \ln n - n$):

$$h \geq \sum_{k=1}^{n} \log_2 k \geq \sum_{k=n/2}^{n} \log_2 \left(\frac{n}{2}\right) = \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \square(\mathbf{n} \log_2 \mathbf{n}) \tag{4}$$

Thus, no comparison-based sorting algorithm can achieve a worst-case time complexity faster than $\square(\mathbf{n} \log \mathbf{n})$.

2. Elementary Sorting Algorithms

© Biraj

2.1 Selection Sort

- **Working Principle:** In each pass $i \in [0, n-2]$, find the minimum element in unsorted subarray $A[i \dots n-1]$ and swap it with $A[i]$.
- **Comparisons:** Total comparisons are fixed for all input distributions:

$$C(n) = \sum_{i=0}^{n-2}(n-1-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \square(\mathbf{n^2}) \tag{5}$$

- **Swaps:** At most $n-1$ swaps ($\mathcal{O}(n)$ writes) — ideal when write operations are physically expensive (e.g., EEPROM/Flash memory).
- **Properties:** In-place ($\mathcal{O}(1)$ space), **Unstable** (e.g., sorting $[4_a, 4_b, 2]$ swaps 4_a with 2, placing 4_a after 4_b).

2.2 Bubble Sort

- **Working Principle:** Iteratively scan the array, comparing adjacent pairs $(A[j], A[j+1])$ and swapping them if out of order, bubbling the largest item to the end in each pass.
- **Optimization:** Maintain a boolean flag `swapped`. If no swaps occur in a full pass, the array is already sorted and the algorithm terminates immediately.
- **Complexity:** Best-case: $\mathcal{O}(\mathbf{n})$ (with flag on sorted input). Average and Worst-case: $\mathcal{O}(\mathbf{n^2})$ comparisons and swaps.
- **Properties:** In-place ($\mathcal{O}(1)$ space), **Stable**, Adaptive.

2.3 Insertion Sort

- **Working Principle:** Simulates how cards are sorted in hand: incrementally pick $A[i]$ and insert it into its correct relative position within the sorted subarray $A[0 \dots i-1]$ by shifting larger elements one position to the right.
- **Complexity Analysis:**
  - **Best-Case (Sorted Input):** Only 1 comparison per element, 0 shifts $\implies \mathcal{O}(\mathbf{n})$ time.
  - **Worst-Case (Reverse-Sorted Input):** Max shifts: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \implies \mathcal{O}(\mathbf{n^2})$ time.
  - **Average-Case:** $\frac{n(n-1)}{4}$ shifts $\implies \mathcal{O}(\mathbf{n^2})$ time.
- **Properties:** In-place ($\mathcal{O}(1)$ space), **Stable**, Online (can sort incoming streaming data), highly optimal for small arrays ($n \leq 16$).

3. Advanced Divide-and-Conquer & Tree-Based Sorting

© Biraj

3.1 Quick Sort (Partition-Exchange Sort)

Quick Sort Principle and Partitioning Strategy

Quick Sort operates on the **Divide-and-Conquer** paradigm:

1. **Partition:** Select a pivot element P. Rearrange array such that all elements $\leq P$ precede P, and all elements $\geq P$ succeed P. The pivot P settles into its final sorted position q.
2. **Recurse:** Recursively apply Quick Sort to subarrays $A[\text{low} \dots q-1]$ and $A[q+1 \dots \text{high}]$.

Lomuto Partition Algorithm $(A, \textbf{low}, \textbf{high})$**:**

```
pivot = A[high]; i = low - 1;
for j = low to high - 1 do:
    if (A[j] <= pivot) then:
        i = i + 1; swap(A[i], A[j]);
swap(A[i + 1], A[high]);
return (i + 1);
```

Quick Sort Recurrence Relations and Asymptotic Analysis

1. Best-Case Analysis (Balanced Partition): Pivot consistently bisects the array into two equal halves of size $n/2$:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \implies \mathbf{T(n) = \square(n \log_2 n)} \tag{6}$$

2. Worst-Case Analysis (Extremely Unbalanced Partition): Occurs when the array is already sorted or reverse-sorted and the first or last element is chosen as pivot, yielding subproblems of size 0 and $n-1$:

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + cn = c\sum_{i=1}^{n} i \implies \mathbf{T(n) = \square(n^2)} \tag{7}$$

3. Average-Case Analysis: Assuming all permutations equally probable: $\mathcal{O}(\mathbf{n} \log \mathbf{n})$.

Auxiliary Space: $\mathcal{O}(\log n)$ stack frames for balanced recursion; degrades to $\mathcal{O}(n)$ in worst case. Quick Sort is **Unstable**.

3.2 Merge Sort

Merge Sort Algorithm and Guaranteed Bounds

Merge Sort recursively divides the array into two equal halves until single-element subarrays are reached, then merges them in sorted order using an auxiliary buffer:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \quad \forall n > 1, \quad T(1) = 0 \tag{8}$$

By the Master Theorem (Case 2: $a = 2, b = 2, f(n) = \Theta(n) = \Theta(n^{\log_2 2})$):

$$\mathbf{T(n) = \square(n \log_2 n)} \quad \text{in Best, Average, and Worst Cases!} \tag{9}$$

© Biraj

Two-Way Merge Algorithm $(A, \textbf{low}, \textbf{mid}, \textbf{high})$**:**

```
i = low; j = mid + 1; k = low;
while (i <= mid && j <= high) {
    if (A[i] <= A[j]) { B[k] = A[i]; i++; }
    else { B[k] = A[j]; j++; }
    k++;
}
while (i <= mid) { B[k] = A[i]; i++; k++; }
while (j <= high) { B[k] = A[j]; j++; k++; }
for (p = low; p <= high; p++) A[p] = B[p];
```

Key Trade-offs:
- **Guaranteed Performance:** Never degrades to $\mathcal{O}(n^2)$.
- **Space Overhead:** Requires $\mathcal{O}(\mathbf{n})$ auxiliary memory array.
- **Stability: Stable** (ideal for complex multi-attribute sorting).
- **Linked Lists:** Does not require random access; sorts linked lists in $\mathcal{O}(n \log n)$ time with $\mathcal{O}(1)$ extra space.

3.3 Heap Sort

A **Binary Heap** is a Complete Binary Tree mapped sequentially into an array $A[0 \dots n-1]$ satisfying the Max-Heap property: $A[\text{parent}] \geq A[\text{child}]$.

- Array Index Mapping: $\text{Parent}(i) = \lfloor (i-1)/2 \rfloor$, $\text{Left}(i) = 2i+1$, $\text{Right}(i) = 2i+2$.

Heap Sort Procedures and Linear Build-Heap Proof

1. Max-Heapify(A, i, n) $(\mathcal{O}(\log n))$**:**

```
largest = i; left = 2*i + 1; right = 2*i + 2;
if (left < n && A[left] > A[largest]) largest = left;
if (right < n && A[right] > A[largest]) largest = right;
if (largest != i) {
    swap(A[i], A[largest]);
    Max-Heapify(A, largest, n);
}
```

2. Build-Max-Heap $(O(n))$**:** Calls `Max-Heapify` on all internal nodes from $\lfloor n/2 \rfloor - 1$ down to 0:

$$\sum_{h=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \mathcal{O}(h) = \mathcal{O}\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = \mathcal{O}(n \cdot 2) = \mathcal{O}(\mathbf{n}) \tag{10}$$

3. Heap Sort Algorithm $(\mathcal{O}(n \log n))$**:**
1. Build a Max-Heap from input array: $\mathcal{O}(n)$.
2. For $i = n-1$ down to 1:
   - Swap root $A[0]$ (current maximum) with $A[i]$.
   - Reduce heap size by 1.
   - Call `Max-Heapify`$(A, 0, i)$: $\mathcal{O}(\log n)$.

Complexity: Best, Average, Worst time strictly $\mathcal{O}(\mathbf{n} \log \mathbf{n})$. In-place $(\mathcal{O}(\mathbf{1})$ space). **Unstable**.

3.4 Comprehensive Sorting Comparison Matrix

Algorithm	Best	Average	Worst	Space	Stable?	Primary Use Case
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	**No**	Minimal write operations.
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	**Yes**	Educational; nearly sorted data.
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	**Yes**	Small lists ($n \leq 16$); streaming.
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	**No**	Fastest general in-memory sort.
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	**Yes**	Linked lists; external sorting.
Heap Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	**No**	Guaranteed $\mathcal{O}(n \log n)$ in-place.

4. Hashing Techniques and Collision Resolution

Definition: Hashing, Hash Table and Load Factor

Hashing is a transformation technique that maps large variable keys K from a universe U into fixed integer slot indices in a **Hash Table** $T[0 \dots m-1]$ via a mathematical **Hash Function** $h(K)$:

$$h : U \longrightarrow \{0, 1, \dots, m-1\} \tag{11}$$

The **Load Factor** (α) is the ratio of stored keys n to table capacity m:

$$\alpha = \frac{n}{m} \tag{12}$$

A **Collision** occurs when two distinct keys hash to the same slot: $h(K_1) = h(K_2)$ for $K_1 \neq K_2$.

4.1 Standard Hash Functions

1. **Division Method:** $h(K) = K \bmod m$. The table size m should be a prime number not close to a power of 2 or 10 to minimize clustering.
2. **Mid-Square Method:** Square the key K^2, and extract r middle digits/bits as the index. Table size $m = 2^r$.
3. **Folding Method:** Partition K into parts matching index digit length, then sum parts:
   - **Fold-Shift:** Add components directly: $h(K) = (p_1 + p_2 + \dots) \bmod m$.
   - **Fold-Boundary:** Reverse the digits of the outer parts before adding.
4. **Multiplication Method:** $h(K) = \lfloor m(KA \bmod 1) \rfloor$, where $0 < A < 1$ (Knuth recommends $A \approx \frac{\sqrt{5}-1}{2} \approx 0.618033$).

© Biraj

4.2 Collision Resolution Techniques

1. **Separate Chaining (Open Hashing):** Each table slot is a header pointer to a linked list of records hashing to that slot.
   - Can accommodate $\alpha > 1$ (table never gets "full").
   - Average search time: $\mathcal{O}(1 + \alpha)$.
2. **Open Addressing (Closed Hashing):** All records reside directly in the table array ($m \geq n \implies \alpha \leq 1$). If collision occurs, search alternate slots via a systematic probe sequence $h(K, i)$:
   - **Linear Probing:** $h(K, i) = (h'(K) + i) \bmod m$. Suffers from **Primary Clustering** (adjacent occupied slots merge into long chains, degrading search to $\mathcal{O}(n)$).
   - **Quadratic Probing:** $h(K, i) = (h'(K) + c_1 i + c_2 i^2) \bmod m$. Eliminates primary clustering but suffers from **Secondary Clustering** (keys hashing to same initial slot trace identical probe paths).
   - **Double Hashing:** $h(K, i) = (h_1(K) + i \cdot h_2(K)) \bmod m$. Probe offset depends on the key itself ($h_2(K)$). Completely eliminates both primary and secondary clustering! Condition: $\gcd(h_2(K), m) = 1$.

Exam Numerical: Collision Resolution Walkthrough

Problem: Insert keys $K = \{23, 43, 13, 27\}$ into a hash table of size $m = 7$ using $h'(K) = K \bmod 7$.

1. Under Linear Probing ($h(K, i) = (h'(K) + i) \bmod 7$)**:**

- $K = 23$: $h'(23) = 23 \bmod 7 = 2$. Slot 2 is empty $\implies T[2] = 23$.
- $K = 43$: $h'(43) = 43 \bmod 7 = 1$. Slot 1 is empty $\implies T[1] = 43$.
- $K = 13$: $h'(13) = 13 \bmod 7 = 6$. Slot 6 is empty $\implies T[6] = 13$.
- $K = 27$: $h'(27) = 27 \bmod 7 = 6$. **Collision at 6!** Probe $i = 1$: $(6 + 1) \bmod 7 = 0$. Slot 0 is empty $\implies T[0] = 27$.

Final Table: $T[0] = 27,\ T[1] = 43,\ T[2] = 23,\ T[6] = 13$.

5. Graph Theory, Representations and Traversals

Definition: Graph Data Structure and Core Terminologies

A **Graph** $G = (V, E)$ consists of a non-empty finite set of vertices V and a set of edges E connecting pairs of vertices.

- **Handshaking Lemma:** In any undirected graph:

$$\sum_{v \in V} \deg(v) = 2|E| \tag{13}$$

  Corollary: In any graph, the number of vertices with odd degree is always even.
- **Complete Graph** K_n**:** Every pair of distinct vertices is connected by a unique edge: $|E| = \frac{n(n-1)}{2}$.
- **Connected Component:** A maximal connected subgraph of an undirected graph.

© Biraj

- **Strongly Connected Digraph:** A directed graph where there exists a directed path between every ordered pair of vertices (u, v).

5.1 Graph Storage Representations

Representation	Space Complexity	Edge Lookup (u, v)	Optimal Suitability
Adjacency Matrix	$\mathcal{O}(V^2)$	$\mathcal{O}(1)$	Dense graphs ($\|E\| \approx \|V\|^2$); fast edge tests.
Adjacency List	$\mathcal{O}(V + E)$	$\mathcal{O}(\deg(u))$	Sparse graphs ($\|E\| \ll \|V\|^2$); DFS/BFS algorithms.
Incidence Matrix	$\mathcal{O}(V \cdot E)$	$\mathcal{O}(E)$	Theoretical network flows; hypergraphs.

5.2 Graph Search and Traversal Algorithms

Breadth-First Search (BFS) and Depth-First Search (DFS) Algorithms

1. Algorithm BFS(G, s):

```
initialize boolean visited[V] = false, Queue Q;
visited[s] = true; Enqueue(Q, s);
while (!IsEmpty(Q)) {
    u = Dequeue(Q); print(u);
    for each vertex v adjacent to u:
        if (!visited[v]) {
            visited[v] = true; Enqueue(Q, v);
        }
}
```

Time: $\mathcal{O}(\mathbf{V} + \mathbf{E})$ (Adjacency List), $\mathcal{O}(V^2)$ (Matrix) Space: $\mathcal{O}(V)$

2. Algorithm DFS(G, u):

```
visited[u] = true; print(u);
for each vertex v adjacent to u:
    if (!visited[v]) {
        DFS(G, v); // Recursive tree exploration
    }
```

Time: $\mathcal{O}(\mathbf{V} + \mathbf{E})$ Space: $\mathcal{O}(V)$ (System recursion stack)

Edge Classification in DFS: Tree edges (forming DFS forest), Back edges (to ancestor $\implies$ cycle detected!), Forward edges (to descendant), Cross edges (between distinct subtrees).

© Biraj

Quick Revision — Module 4

Sorting, Hashing and Graph Theory | ES-CS301

Key Formulas, Recurrences and Complexities at a Glance

Comparison Sort Lower Bound	$\Omega(n \log_2 n)$ via Decision Tree Height $h \geq \log_2(n!)$
Merge Sort Recurrence	$T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log_2 n)$ (Guaranteed)
Quick Sort Recurrences	$T_{\text{avg}}(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log_2 n)$, $T_{\text{worst}}(n) = \Theta(n^2)$
Linear Build-Heap Bound	$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} \mathcal{O}(h) = \mathcal{O}(n)$ (Linear Time Proof)
Hash Table Load Factor	$\alpha = n/m$ ($\alpha \leq 1$ for open addressing, $\alpha > 1$ for chaining)
Double Hashing Probe Offset	$h(K, i) = (h_1(K) + i \cdot h_2(K)) \bmod m$ ($h_2(K)$ coprime to m)
Graph Handshaking Lemma	$\sum_{v \in V} \deg(v) = 2\lvert E \rvert \implies$ Number of odd-degree vertices is even
BFS / DFS Traversal Time	$\mathcal{O}(V+E)$ (Adjacency List), $\mathcal{O}(V^2)$ (Adjacency Matrix)

One-Line Definitions

- **Stable Sort:** A sorting algorithm that strictly preserves the relative input order of records with duplicate keys.
- **In-Place Sort:** A sorting algorithm requiring strictly $\mathcal{O}(1)$ auxiliary memory space beyond the input buffer.
- **Max-Heap:** A complete binary tree where key of every parent node is greater than or equal to its children ($A[\text{parent}(i)] \geq A[i]$).
- **Primary Clustering:** The undesirable tendency in linear probing where occupied adjacent slots coalesce into large continuous blocks.
- **Double Hashing:** An open addressing collision resolution technique where probe step size is calculated by an independent secondary hash function.
- **Adjacency List:** An array of linked lists where list i stores all adjacent neighbors of vertex i, requiring optimal $\mathcal{O}(V + E)$ space.
- **Breadth-First Search (BFS):** Concentric layer-by-layer graph traversal implemented via a FIFO queue; yields shortest paths in unweighted graphs.
- **Depth-First Search (DFS):** Deep branch backtracking traversal using recursion/stack; detects cycles definitively via back edges.

© Biraj

Frequently Asked / Viva Questions (Part 1)

Q1. Why can comparison-based sorting algorithms not execute faster than $\mathcal{O}(n \log n)$ in the worst case?

Every comparison sort can be modeled as a decision tree with $n!$ leaves corresponding to all permutations. A binary tree of height h has at most 2^h leaves. Therefore, $2^h \geq n! \implies h \geq \log_2(n!) \approx n \log_2 n - n \log_2 e = \Omega(n \log n)$. The height represents the worst-case number of comparisons.

Q2. Under what conditions does Quick Sort degrade to its worst-case $\mathcal{O}(n^2)$ time?

When the partition is maximally unbalanced (e.g., one subproblem of size $n-1$ and one of size 0). This occurs when the array is already sorted or reverse-sorted and the first or last element is consistently selected as the pivot.

Q3. Why is Heap construction (`Build-Max-Heap`) $\mathcal{O}(n)$ rather than $\mathcal{O}(n \log n)$?

Although `Max-Heapify` takes $\mathcal{O}(h)$ time, most nodes in a binary tree reside near the leaves with very small heights h. Summing the cost across all heights $\sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} h = \frac{n}{2} \sum \frac{h}{2^h} = \frac{n}{2} \cdot 2 = \mathcal{O}(n)$.

Q4. Compare Separate Chaining and Open Addressing for collision resolution.

Separate Chaining uses external linked lists for colliding elements; it never suffers from table overflow and handles load factor $\alpha > 1$. Open Addressing stores all elements directly within the array table ($\alpha \leq 1$); it eliminates pointer memory overhead but requires collision probing and can suffer from clustering.

Q5. What is the difference between Primary Clustering and Secondary Clustering?

Primary clustering occurs in linear probing when different keys hashing to adjacent slots merge into a single long contiguous chain. Secondary clustering occurs in quadratic probing when two distinct keys having the same initial hash index trace the exact same probe sequence.

Q6. State the Handshaking Lemma and explain its primary corollary.

The Handshaking Lemma states that in any undirected graph, $\sum_{v \in V} \deg(v) = 2|E|$, because each edge contributes 1 degree to two incident vertices. The corollary is that the number of vertices with odd degree in any graph must be strictly even.

Q7. Compare Adjacency Matrix and Adjacency List representations of a graph.

Adjacency Matrix uses $\mathcal{O}(V^2)$ space and checks edge existence in $\mathcal{O}(1)$ time; it is optimal for dense graphs ($|E| \approx |V|^2$). Adjacency List uses $\mathcal{O}(V+E)$ space and finds neighbors in $\mathcal{O}(\deg(v))$ time; it is optimal for sparse graphs ($|E| \ll |V|^2$).

© Biraj

Frequently Asked / Viva Questions (Part 2)

Q8. How does DFS detect cycles in a directed graph?

During DFS traversal, vertices are colored White (unvisited), Gray (in active recursion stack), and Black (completely processed). If an edge (u, v) encounters a Gray vertex v, that edge is a **Back Edge**, which definitively indicates the presence of a directed cycle.

Q9. Why is Merge Sort preferred over Quick Sort for sorting linked lists?

In linked lists, random access is not available, which makes Quick Sort partitioning slow. Merge Sort divides a linked list in $\mathcal{O}(n)$ using the slow-fast pointer technique, and merges two sorted lists in $\mathcal{O}(n)$ time and strictly $\mathcal{O}(1)$ auxiliary space without requiring contiguous arrays.

Q10. What is Topological Sorting, and what type of graph is required?

Topological sorting is a linear ordering of vertices such that for every directed edge (u, v), vertex u appears before v. It is strictly defined only on a **Directed Acyclic Graph (DAG)**.

Summary Matrix: Comprehensive Comparison of Sorting, Hashing and Graphs

Technique / Topic	Core Mechanism	Time / Space Complexity	Key Practical Application
Quick Sort	**Divide-and-Conquer; Pivot partitioning**	**Avg:** $\mathcal{O}(n \log n)$, **Worst:** $\mathcal{O}(n^2)$ **Space:** $\mathcal{O}(\log n)$ **stack**	Fastest general in-memory primitive data sorting.
Merge Sort	**Divide-and-Conquer; Two-finger merge**	**Best/Avg/Worst:** $\mathcal{O}(n \log n)$ **Space:** $\mathcal{O}(n)$ **auxiliary**	External storage sorting, linked list sorting.
Heap Sort	**Binary Max-Heap; Root-leaf exchange**	**Guaranteed** $\mathcal{O}(n \log n)$ **all cases Space:** $\mathcal{O}(1)$ **in-place**	Systems where guaranteed $\mathcal{O}(n \log n)$ and minimal memory are required.
Double Hashing	**Secondary hash step;** $h_1(K) + i \cdot h_2(K)$	**Search:** $\mathcal{O}(1)$ **avg, Worst:** $\mathcal{O}(m)$ **Space:** $\mathcal{O}(m)$ **array**	Hash tables requiring zero primary/secondary clustering.
BFS	**FIFO Queue; Level-order expansion**	$\mathcal{O}(V + E)$ **(with Adj List) Space:** $\mathcal{O}(V)$ **queue**	Shortest paths in unweighted graphs, web crawlers.
DFS	**LIFO Stack / Recursion; Deep path exploration**	$\mathcal{O}(V + E)$ **(with Adj List) Space:** $\mathcal{O}(V)$ **stack**	Cycle detection, topological sorting, connected components.

© Biraj